1 / 8

Keep keys and values for later tokens.

A model-managed cache preserves earlier attention inputs.

> Model-managed tensors are separate from hardware caches.

Synthetic decoder budget

Stored keys	Stored values
512 tokens per sequence	512 tokens per sequence

Layers / batch	**32 / 1**
Parameters	**7 billion**
Weight and cache storage	**2 bytes / value**
Cache width / hidden width	**1,024 / 4,096**

Separate from the opening four-component PyTorch calculation

Copyright © 2026 Christopher Snow

2 / 8

Count the existing cache payload.

Count keys and values across layers, sequences and cached tokens.

> 1 MiB = 1,048,576 bytes; 1 KiB = 1,024 bytes.

Existing key and value payload

Keys and values	**2 tensors**
Decoder layers	**32**
Independent sequences	**1**
Cached tokens / sequence	**512**
Components / key or value	**1,024**
Storage / value	**2 bytes**

64 MiB = 67,108,864 bytes

Multiply all six factors · occupied payload, excluding spare slots

Copyright © 2026 Christopher Snow

3 / 8

Decode appends one new token.

Read the previous cache and write one new entry per sequence.

> One invocation processes one new token per sequence.

One new token after the existing cache

Tokens processed	**1**
Cache tokens / sequence	**512 → 513**
Previous cache read	**64 MiB**
New cache write	**128 KiB**
Weight read	**14 GB**
Arithmetic operations	**0.014268 trillion**

Ideal one weight read / invocation · selected transfers only

Copyright © 2026 Christopher Snow

4 / 8

Prefill starts with an empty cache.

Process the whole prompt and build its cache entries.

> Full square attention count; no causal triangular savings.

Whole prompt from an empty cache

Tokens processed	**512**
Cache tokens / sequence	**0 → 512**
Previous cache read	**0 MiB**
New cache write	**64 MiB**
Weight read	**14 GB**
Arithmetic operations	**7.305439 trillion**

Ideal one weight read / invocation · selected transfers only

Copyright © 2026 Christopher Snow

5 / 8

Compare two ideal time bounds.

Divide counted bytes and operations by the stated rates.

> Hidden width 4,096 · rates and arithmetic are synthetic.

1,000 GB/s bandwidth · 100 TFLOP/s compute

Decimal rates · 1 TFLOP/s = 10^{12} operations per second

Decode

14.067 ms memory

0.143 ms compute

Take the maximum

14.067 ms

Prefill

14.067 ms memory

73.054 ms compute

Take the maximum

73.054 ms

Ideal lower bounds · no measurement supplied

Bytes include weights + old cache reads + new cache writes

Copyright © 2026 Christopher Snow

6 / 8

A batch can share a weight read.

Four sequences produce four new tokens in one invocation.

> Each batch member is an independent sequence.

Decode · same 512 cached tokens per sequence

Batch 1	Batch 4
1 new token	4 new tokens
14 GB weights read	14 GB weights read
64 MiB old cache	256 MiB old cache
14.067 ms / invocation	14.269 ms / invocation
14.067 ms / token	**3.567 ms / token**

Amortised ideal budget · not request latency or measured throughput

Copyright © 2026 Christopher Snow

7 / 8

Pause: make the context four times longer.

Keep four sequences, the same weights and the same assumed rates.

> Distinguish the old payload from the appended entries.

Batch 4 decode · change only previous cache length

Independent sequences	**4**
Cached tokens / sequence	**512 → 2,048**
Weight payload	**14 GB**
Layers / cache width	**32 / 1,024**
Cache storage / value	**2 bytes**

Cache payload: ? · invocation lower bound: ?

Same 1,000 GB/s and 100 TFLOP/s · predict before comparing

Synthetic decoder · ideal weight reuse · selected transfers · no measured GPU time

Copyright © 2026 Christopher Snow

8 / 8

Larger capacity does not scale every cost.

The cache quadruples; the weight read and appended entries stay fixed.

> A real bottleneck needs evidence from the actual implementation.

Four decode sequences · longer previous cache

512 cached tokens → 2,048 cached tokens

Existing cache payload	**256 MiB → 1,024 MiB**
New cache write	**512 KiB (unchanged)**
Weight read	**14 GB (unchanged)**
New compute bound	**0.603 ms**
Invocation lower bound	**14.269 → 15.074 ms**

Recompute max(memory, compute) · capacity and time have different scopes

Synthetic decoder · ideal weight reuse · selected transfers · no measured GPU time

Copyright © 2026 Christopher Snow